

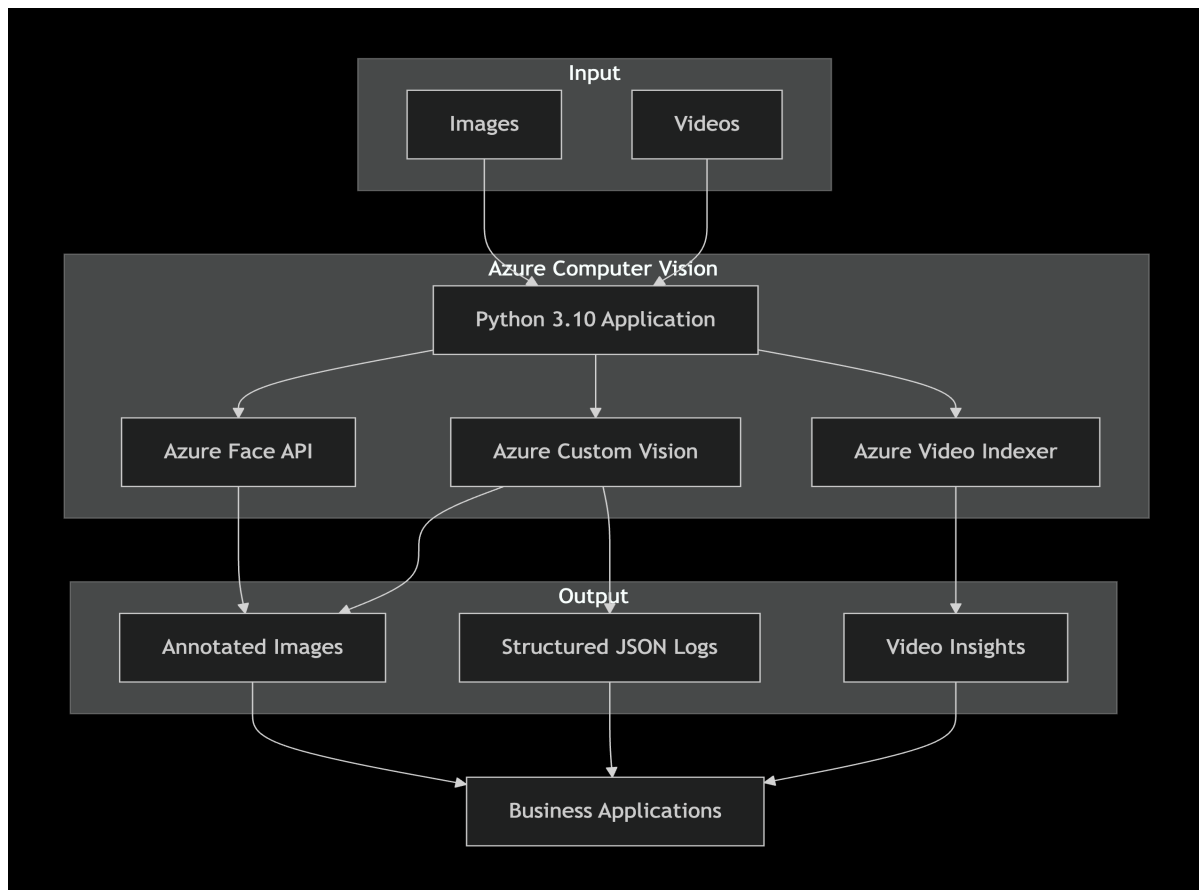
ANASIEZE IKENNA – CLOUD & AI ENGINEER

Project: Azure-Powered Computer Vision: Automated Image Analysis & Object Detection

Overview

I designed an AI-powered image analysis tool using Azure Vision APIs to automate structured insight extraction from images — reducing manual tagging and accelerating decision-making across visual data workflows.

Architecture



Problem

Working with raw image data locally often results in:

- Manual tagging and annotation that doesn't scale
- Limited access to advanced computer vision without complex setups
- Risk of inconsistent results using oGline tools
- Slow workflows that delay insight extraction

Goal: Create a lightweight, reproducible tool that users can use to:

- Analyze images using pre-trained and custom models
- Perform OCR to read text from images
- Detect and recognize human faces and emotions
- Train custom models for image classification and object detection
- Analyze video content and generate AI-powered insights

Tech Stack

Component	Tools Used
Programming	Python 3.10
SDK/API	Azure AI Vision SDK (azure-aivisionimageanalysis)
Image Handling	PIL, Matplotlib
Security	dotenv for credentials
Output Format	Visual overlays (.jpg), structured logs (stdout)

Key Features

Detect, Analyze, and Recognize Faces

- Face detection with bounding boxes
- Attribute analysis (age, emotion, pose)
- Face verification & identification
- Facial landmarks (eyes, nose, mouth)

Classify Images (Custom Vision)

- Custom classifier training
- High accuracy with few images
- Easy retraining & versioning
- Export to edge or cloud
- API + UI integration

Detect Objects in Images

- Object detection with boxes
- Custom labels supported
- Real-time detection ready
- Incremental retraining
- Deploy anywhere

Analyze Video (Video Indexer)

- Scene segmentation
- Face tracking across clips
- Text from video frames
- Speaker indexing + transcript
- AI-generated video insights

A) Face Detection, Analysis and Recognition

Built applications that can detect human faces, analyze facial attributes, and recognize individuals using Azure's AI Vision Face API. Facial analysis has applications in:

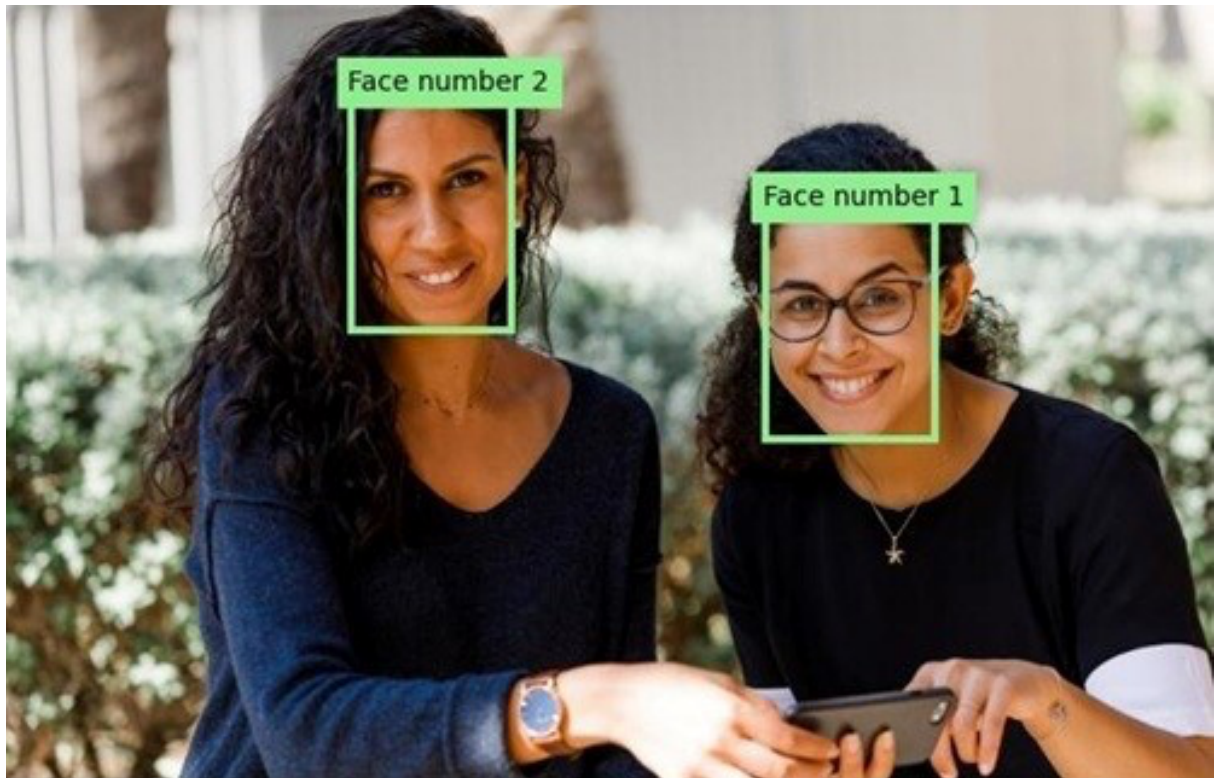
- Identity verification systems (e.g. KYC)
- Facial unlock and personalized user experiences
- Surveillance and access control in smart buildings
- Retail customer sentiment analysis

Use Case: Human Face Detection



- Configure env variables
- Run **analyze-faces.py**

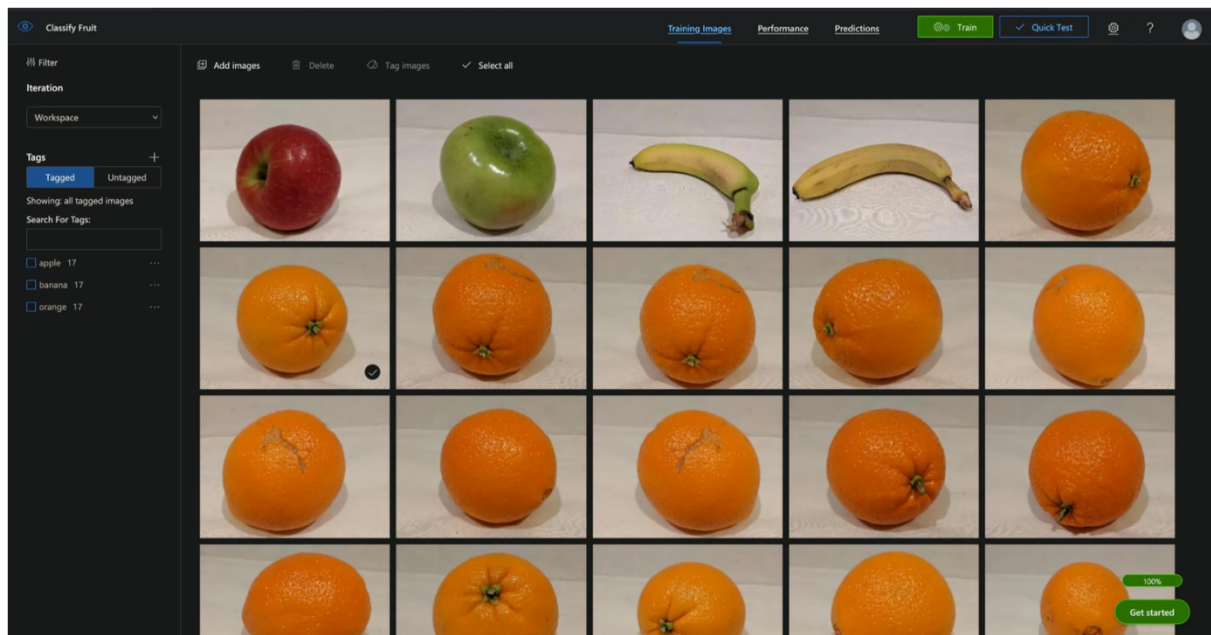
B) Classifying Images



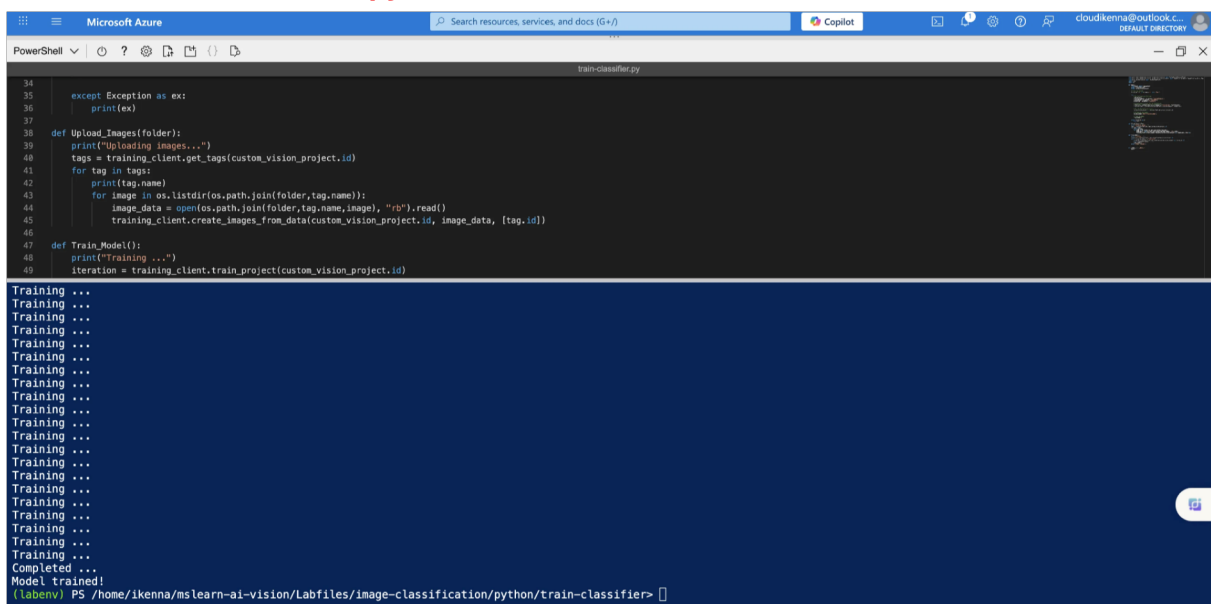
I designed a system that classify images based on custom categories by training models with Azure Custom Vision. Image classification has applications in:

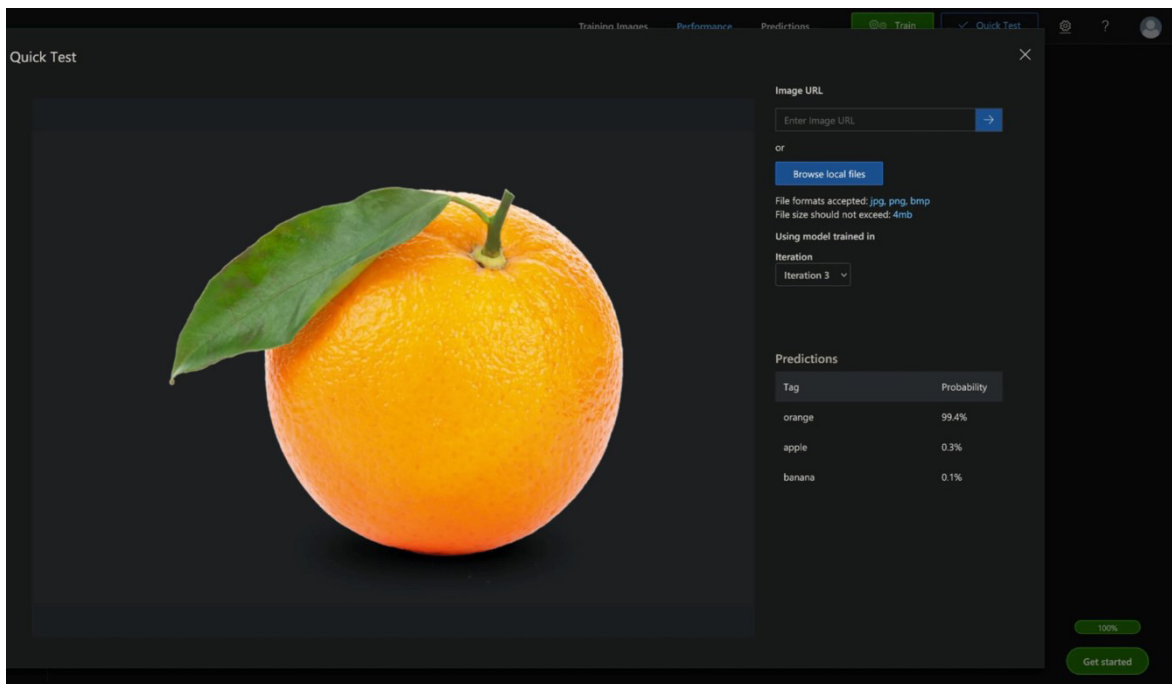
- Retail: Automatically tagging products for inventory management
- Agriculture: Identifying plant diseases or animal breeds
- Construction: Classifying safety gear in site photos
- Healthcare: Recognizing specific medical conditions in images (with proper regulatory guidance)
- Digital Media: Categorizing images for content moderation or recommendations

Use Case: Training Model for Fruit Classification



- Configure env variables
- Run **code_train.py** to train the models





- Configure env variables
- Run **code_classify.py** to classify the data

```

13 load_dotenv()
14 prediction_endpoint = os.getenv('PredictionEndpoint')
15 prediction_key = os.getenv('PredictionKey')
16 project_id = os.getenv('ProjectID')
17 model_name = os.getenv('ModelName')
18
19 # Authenticate a client for the prediction API
20 credentials = ApiKeyCredentials(in_headers={"Prediction-key": prediction_key})
21 prediction_client = CustomVisionPredictionClient(endpoint=prediction_endpoint, credentials=credentials)
22
23 # Classify test images
24 for image in os.listdir('test-images'):
25     image_data = open(os.path.join('test-images', image), "rb").read()
26     results = prediction_client.classify_image(project_id, model_name, image_data)
27
28     # Loop over each label prediction and print any with probability > 50%
29     for prediction in results.predictions:
30         if prediction.probability > 0.5:
31             print(image, ': {} ( {:.0%} )'.format(prediction.tag_name, prediction.probability))
32 except Exception as ex:
33     print(ex)

```

IMG_TEST_2.jpg : banana (100%)
 IMG_TEST_3.jpg : orange (100%)
 IMG_TEST_1.jpg : apple (98%)
 (labenv) PS /home/ikenna/mslearn-ai-vision/Labfiles/image-classification/python/test-classifier>

B) Detect Object in Images

Develop models that detect and localize multiple objects within a single image

Real-world applications:

- Retail Loss Prevention: Spot concealed items or unscanned products at checkout

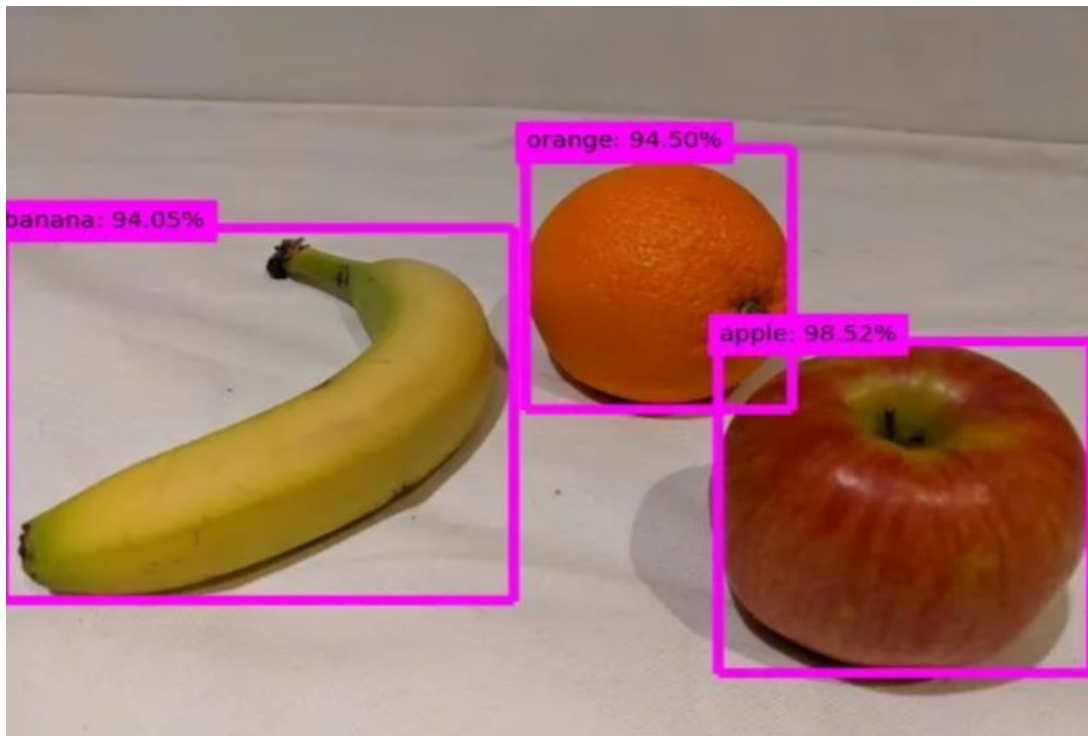
- Smart Construction Monitoring: Detect missing safety gear, blocked exits, or unauthorized vehicles
- Warehouse Automation: Track inventory movement, pallet detection, or forklift usage
- TraGic & Parking Management: Identify license plates, illegal parking, vehicle counts
- Logistics: Detect package types, barcodes, or label presence for sorting automation
- Configure env variables

- Run **train_detector.py** to detect the labels

```
PowerShell
add-tagged-images.py
18 load_dotenv()
19 training_endpoint = os.getenv('TrainingEndpoint')
20 training_key = os.getenv('TrainingKey')
21 project_id = os.getenv('ProjectID')
22
23 # Authenticate a client for the training API
24 credentials = ApiKeyCredentials(in_headers={"Training-key": training_key})
25 training_client = CustomVisionTrainingClient(training_endpoint, credentials)
26
27 # Get the Custom Vision project
28 custom_vision_project = training_client.get_project(project_id)
29
(labenv) PS /home/ikenna/mslearn-ai-vision/Labfiles/object-detection/python/train-detector> ls -a -l
total 40
drwxr-xr-x 4 ikenna ikenna 4096 Aug 14 20:02 .
drwxr-xr-x 4 ikenna ikenna 4096 Aug 14 20:01 ..
-rw-r--r-- 1 ikenna ikenna 2899 Aug 14 20:01 add-tagged-images.py
-rw-r--r-- 1 ikenna ikenna 209 Aug 14 20:03 .env
drwxr-xr-x 2 ikenna ikenna 4096 Aug 14 20:01 images
drwxr-xr-x 5 ikenna ikenna 4096 Aug 14 20:02 labenv
-rw-r--r-- 1 ikenna ikenna 7 Aug 14 20:01 requirements.txt
-rw-r--r-- 1 ikenna ikenna 9990 Aug 14 20:01 tagged-images.json
(labenv) PS /home/ikenna/mslearn-ai-vision/Labfiles/object-detection/python/train-detector> cat ./requirements.txt
dotenv
(labenv) PS /home/ikenna/mslearn-ai-vision/Labfiles/object-detection/python/train-detector>
```

- Configure env variables
- Run **test_detector.py** to test the model performance

```
PowerShell
Detecting objects in produce.jpg
apple
orange
banana
Results saved in output.jpg
(labenv) PS /home/ikenna/mslearn-ai-vision/Labfiles/object-detection/python/test-detector>
(labenv) PS /home/ikenna/mslearn-ai-vision/Labfiles/object-detection/python/test-detector>
```

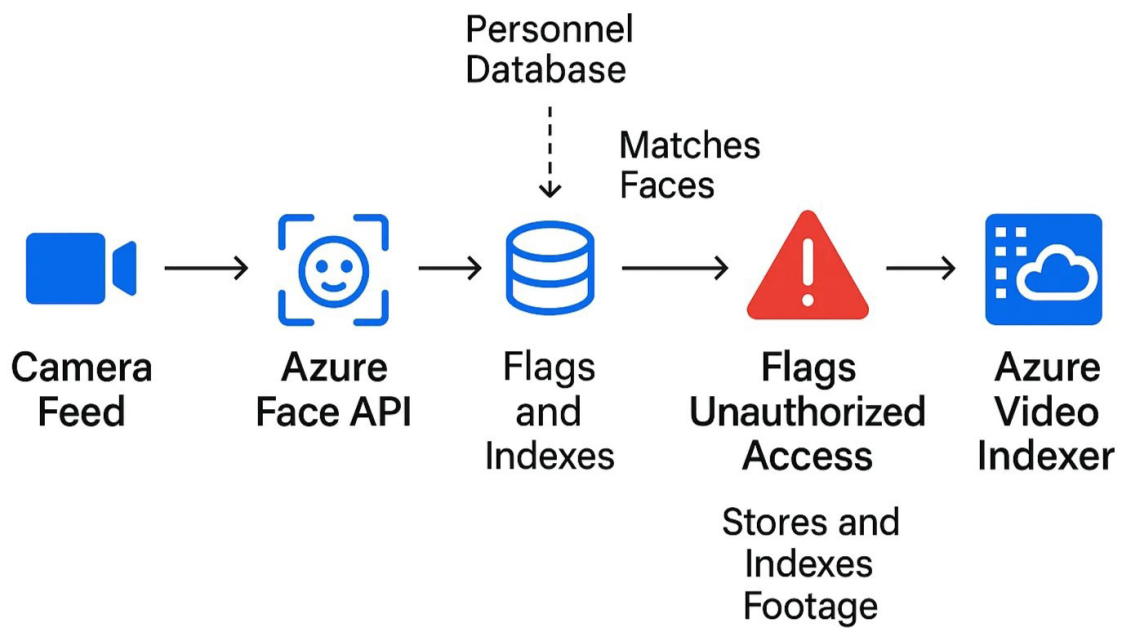
Outcome

- Built cloud-native tools for image tagging, OCR, face detection & object recognition
- Trained and tested custom classification models via Azure Custom Vision
- Extracted insights from video: face tracking, speech-to-text, scene segmentation
- Wrote modular Python scripts using Azure SDKs, dotenv, and PIL
- Produced visual and structured outputs (image overlays, logs, JSON)
- Secured credentials and APIs via **.env** handling

Business Case Usage: Smart Access Control

- Use facial recognition to automate secure building entry and flag unauthorized access in real time.

Tech Stack: Azure Face API · Video Indexer · Python SDK · dotenv



What It Does:

- Detects and identifies faces from camera feeds
- Matches against a list of approved personnel
- Flags unknown individuals for review
- Stores and indexes video footage for audit

Business Impact

- Faster and smarter entry process for authorized staff
- Reduced risk of unauthorized access
- Audit-ready, searchable video logs
- Enhanced security using AI-powered surveillance

What's Next

- Wrap modules into a deployable Streamlit or fast API demo
- Host app using Azure App Service or Container Apps
- Automate image processing with Azure Functions + Blob Triggers